

Renderer benchmark report

Repeated, fixed-duration browser measurements with the result first and the supporting performance, GPU, memory, traffic, and workload evidence below.

PROFILE heavy	RUNS / IMPLEMENTATION 5	MEASURED WINDOW 10 s fixed	WARMUP 5 s	MACHINE arm64 · 14 CPUs	CAPTURED 8/10/2026, 9:16:34 PM
------------------	----------------------------	-------------------------------	---------------	----------------------------	-----------------------------------

RESULT OVERVIEW

BroMetal leads this renderer comparison

On this scene and machine, BroMetal is the strongest renderer result at 54.8 fps. TypeGPU-Antiky is second at 46.1 fps. Three.js is the least performant at 12.1 fps.

Mean frame-rate order across 5 repeated 10-second captures: 1. BroMetal — 54.8 fps · 2. TypeGPU-Antiky — 46.1 fps · 3. WESL — 45.8 fps · 4. TypeGPU — 37.6 fps · 5. Three.js — 12.1 fps.

The sections below show the run-level uncertainty, frame pacing, GPU-process activity, data movement, resource use, and submitted WebGPU work behind that ordering.

STUDY DESIGN

What this benchmark tests

This is a renderer-level comparison of ahead-of-time shader authoring against runtime shader and graph preparation, together with a directional test of GPU-resident work versus CPU-source WebGPU API traffic.

QUESTION 01 AOT versus runtime preparation BroMetal, TypeGPU-Antiky, and WESL ship static WGSL or generated artifacts. TypeGPU resolves its typed graph in the browser; Three.js builds its WebGPU/TSL graph at runtime. Every path still relies on the browser and driver to compile WGSL and create GPU pipelines.	QUESTION 02 GPU-first execution The harness counts CPU-source queue writes, requested GPU→CPU mappings, mapped-write capacity, external-image copies, and GPU-internal copies separately. This tests the claim that renderer work can remain GPU-resident without inventing physical bus traffic.	COMMON SOURCE Pinned Sponza workload All five demos load Georgi Nikolov's webgpu-sponza-demo asset snapshot at commit 1c90e984...694c: 103 primitives, 262,267 triangles, 69 material images, the same fixed camera, and 40.3 MiB of runtime source assets.	PROTOCOL Repeated isolated captures 5 fresh Chrome processes per renderer, 5 s warmup, then 10 s measured at 1440×900. Renderer order rotates by repetition; HTTP and persistent GPU shader caches are disabled.
--	---	---	--

PACKAGE REFERENCE

Library size: shipped and installed

Browser payload measures runtime code delivered to users. Package contents and installed build stack measure the authoring machinery available on this machine, including its resolved production dependency closure.

BROMETAL · 0.17.1		TYPEGPU · 0.11.9		TYPEGPU-ANTIKY · 0.0.0		WESL · 0.7.28		THREE.JS WEBGPU · 0.185.1	
BROWSER GZIP	11.10 KiB	BROWSER GZIP	78.26 KiB	BROWSER GZIP	None shipped	BROWSER GZIP	None shipped	BROWSER GZIP	279.49 KiB
OWN PACKAGE	1015.29 KiB	OWN PACKAGE	961.81 KiB	OWN PACKAGE	92.47 KiB	OWN PACKAGE	716.47 KiB	OWN PACKAGE	22.10 MiB
INSTALLED BUILD STACK	23.70 MiB	INSTALLED BUILD STACK	1.23 MiB	INSTALLED BUILD STACK	18.89 MiB	INSTALLED BUILD STACK	716.47 KiB	INSTALLED BUILD STACK	22.10 MiB
30.11 KiB minified · brometal		287.92 KiB minified · typegpu		AOT/link step excluded from the browser bundle		AOT/link step excluded from the browser bundle		1012.48 KiB minified · three/webgpu	

Own-package and installed-stack values sum logical file bytes from the pinned local installation; nested node_modules are counted once through the resolved production dependency graph. Dev dependencies, filesystem block overhead, demo code, and scene assets are excluded. Browser gzip is a separately bundled public-entry reference. The benchmark's BroMetal-style demo is locally implemented from static WGSL and does not import the published runtime.

VISUAL WORKLOAD RECORD

One scene, five renderer paths

Actual heavy-profile canvas captures use the same 1440x900 presentation crop and fixed camera. The overlay combines the current repeated benchmark means with the package-entry reference above.

BroMetal AOT TYPESCRIPT → STATIC WGSL

MEAN FPS	P95 FRAME	GPU PROCESS	CPU WRITES	LIBRARY GZIP
54.8 fps	25.0 ms	77.3%	4 / frame	11.10 KiB

TypeGPU RUNTIME TYPEGPU RESOLUTION

MEAN FPS	P95 FRAME	GPU PROCESS	CPU WRITES	LIBRARY GZIP
37.6 fps	33.4 ms	74.6%	4 / frame	78.26 KiB

TypeGPU-Antiky AOT TYPEGPU → STATIC ARTIFACTS

MEAN FPS	P95 FRAME	GPU PROCESS	CPU WRITES	LIBRARY GZIP
46.1 fps	26.9 ms	77.2%	4 / frame	None shipped

WESL STATIC WESL LINKING → WGSL

MEAN FPS	P95 FRAME	GPU PROCESS	CPU WRITES	LIBRARY GZIP
45.8 fps	26.8 ms	71.5%	4 / frame	None shipped

Three.js RUNTIME THREE.JS WEBGPU / TSL

MEAN FPS	P95 FRAME	GPU PROCESS	CPU WRITES	LIBRARY GZIP
12.1 fps	100.8 ms	63.5%	327 / frame	279.49 KiB

REPRODUCIBILITY RECORD

Execution environment

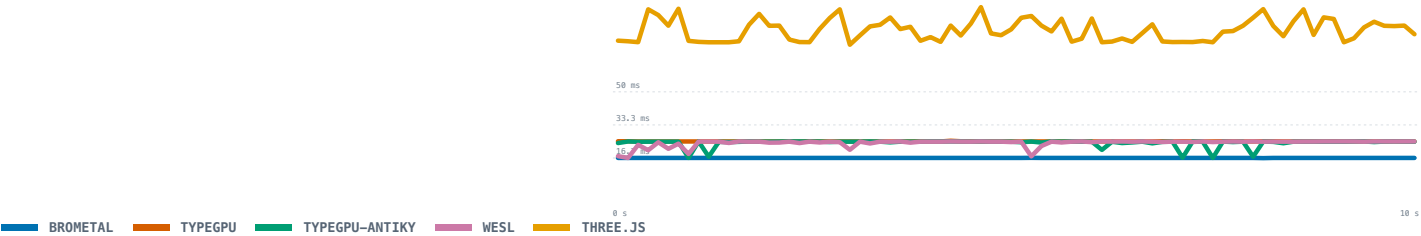
Hardware identity and browser-exposed WebGPU capabilities are captured with every isolated run. Full adapter features and limits remain in the raw result.

WEBGPU ADAPTER apple · metal-3	ADAPTER FEATURES 22 exposed
MAX 2D TEXTURE 16,384 px	MAX BUFFER 4.00 GiB
SYSTEM MEMORY 24.00 GiB	BROWSER Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/151.0.0.0 Safari/537.36

FRAME PACING OVER TIME

Median frame time by elapsed capture time

ALL REPETITIONS · 10 S FIXED WINDOW

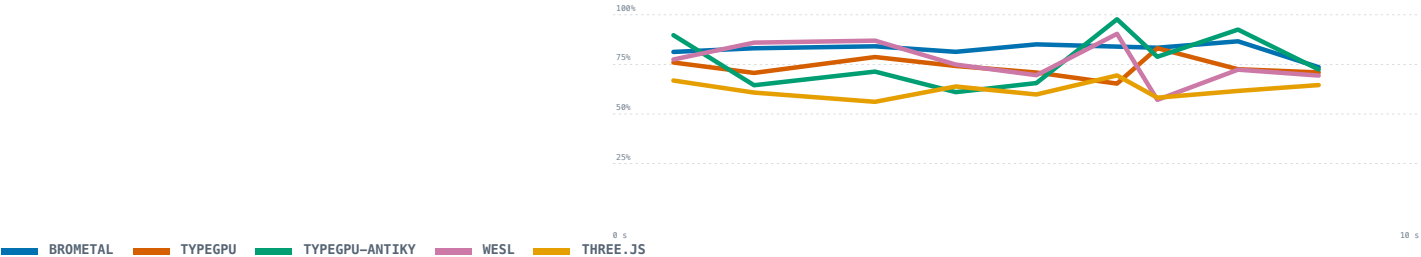


Each point is the median of frame intervals landing in the same elapsed-time bin across repetitions. Tail behavior remains visible in P95 and worst-frame distributions; raw intervals remain in raw.json.

GPU DEMAND OVER TIME

Chrome GPU-process busy intervals

DRIVER-ACCOUNTED · MEDIAN BY ELAPSED-TIME BIN

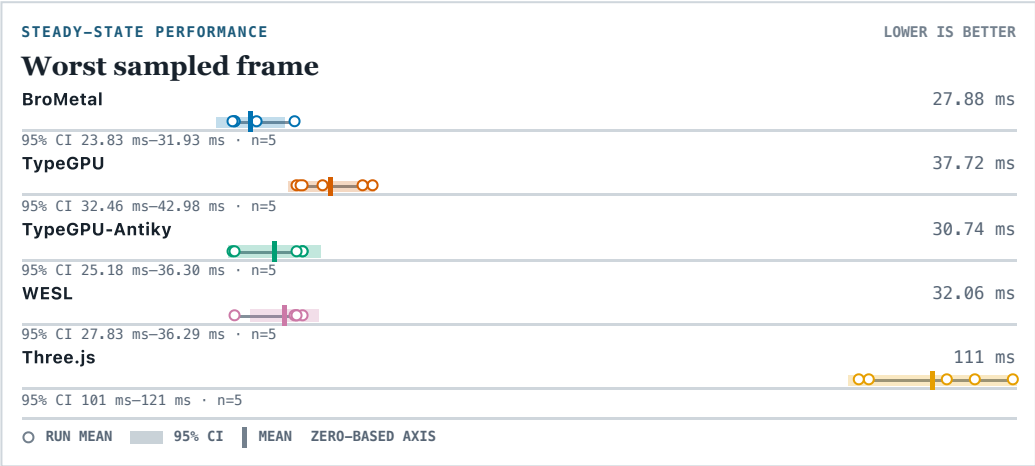
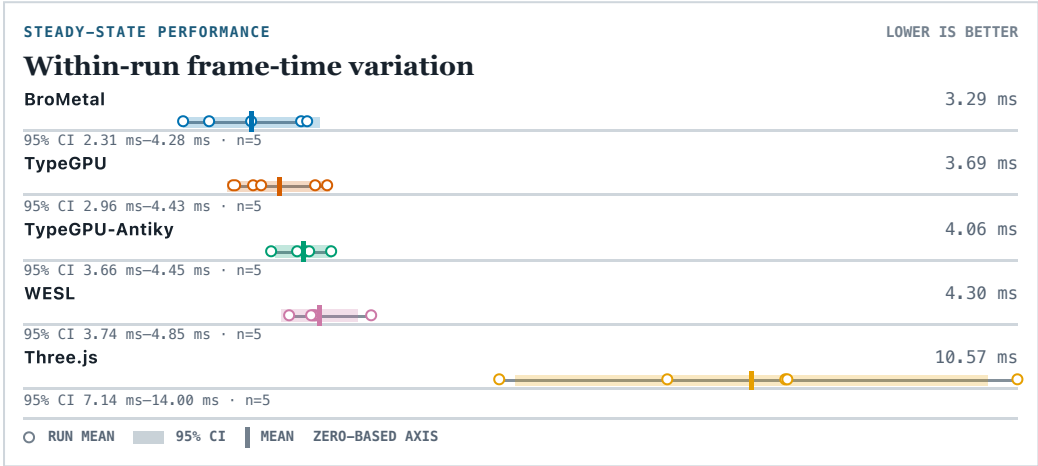
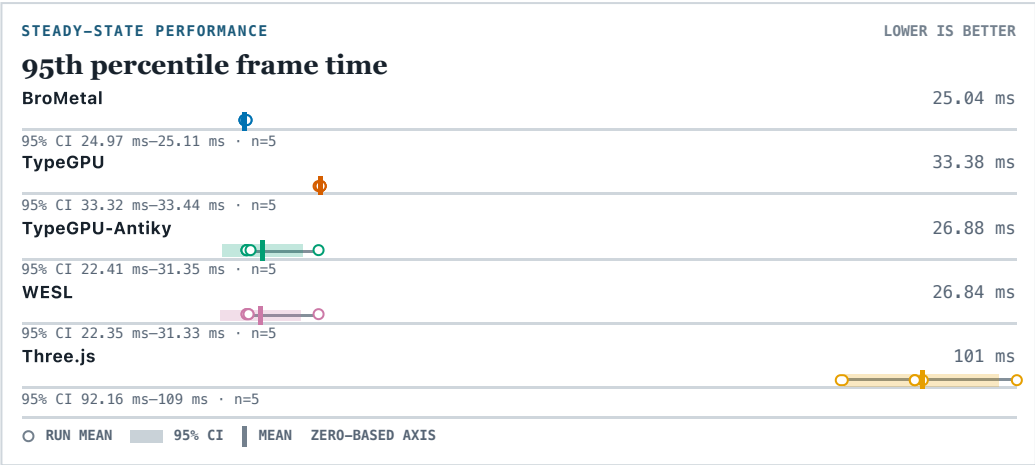
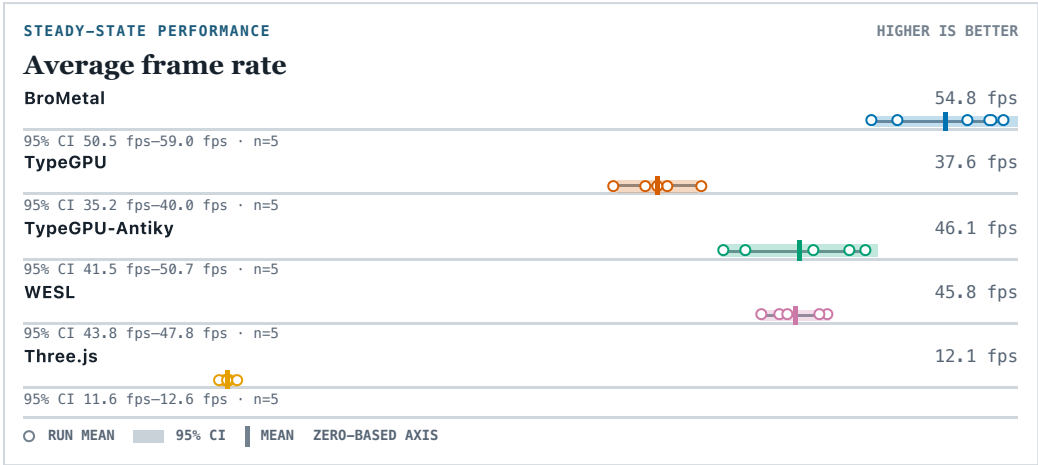


This directional view derives interval utilization from the Chrome GPU process’s cumulative driver time. It is overlaid—not stacked—because renderer series are independent captures and do not form an additive total. It is a demand diagnostic, not portable shader-core occupancy.

MEASURED DISTRIBUTIONS

Steady-state performance

Run-level means and confidence intervals show effect size and repeatability; the timeline shows direction over the capture.



MEASUREMENT BOUNDARY

Data movement, not assumed round trips

This capture classifies WebGPU API activity by direction. It does not infer a CPU↔GPU round trip from a one-way upload or an internal GPU copy.

CPU SOURCE → GPU

queue.writeBuffer and queue.writeTexture. Buffer payload is exact at the API boundary; known texture payload is logical texel/block bytes.

GPU → CPU REQUEST

mapAsync(READ) requested ranges. getMappedRange is counted as access but its bytes are not added again.

EXTERNAL SOURCE → GPU

copyExternalImageToTexture remains separate because video/image decode sources may already be GPU-resident.

GPU-INTERNAL

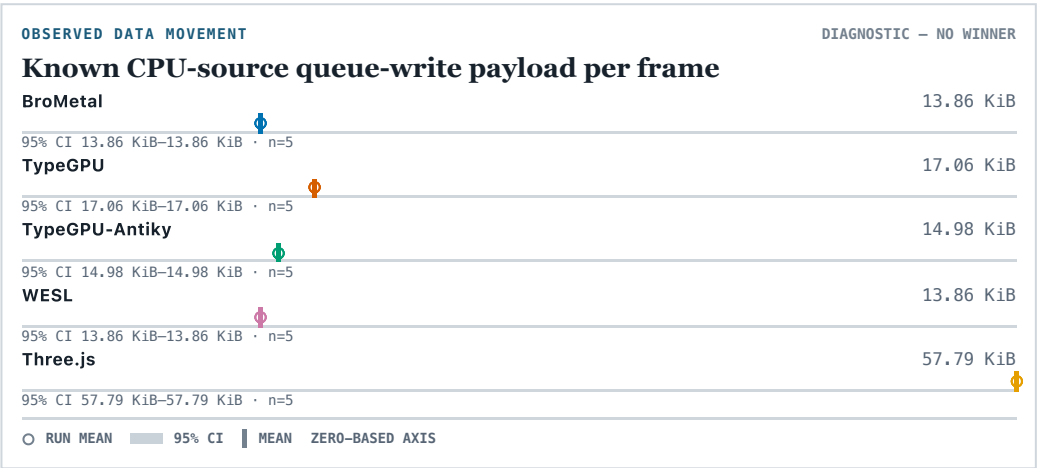
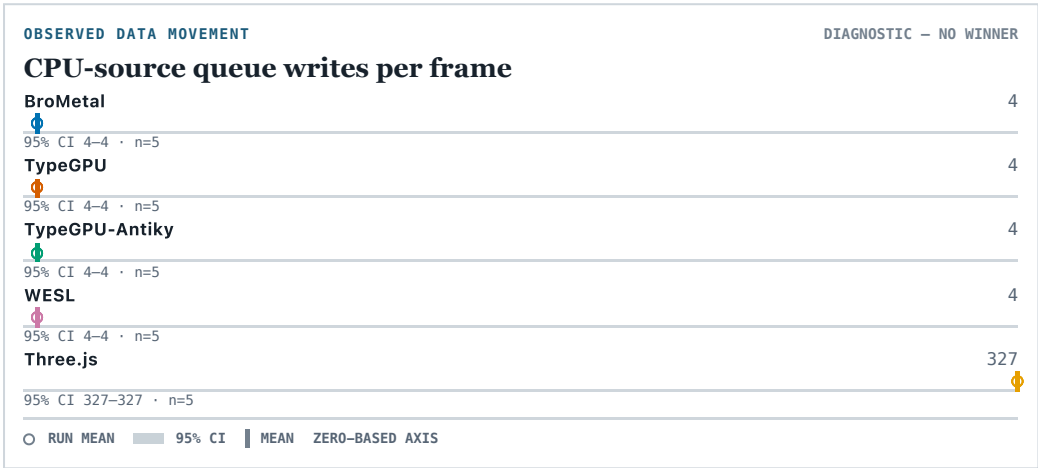
Encoder copy commands are not CPU traffic. Known logical payload and calls describe graph movement only.

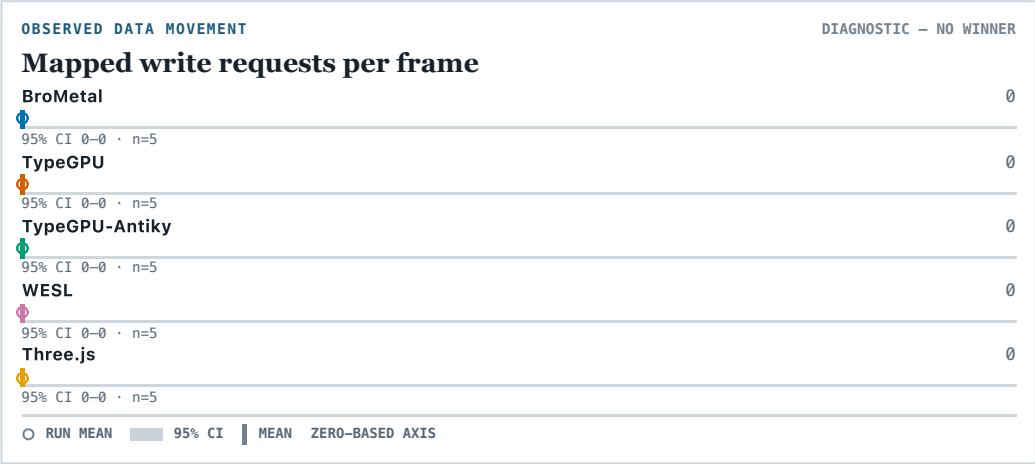
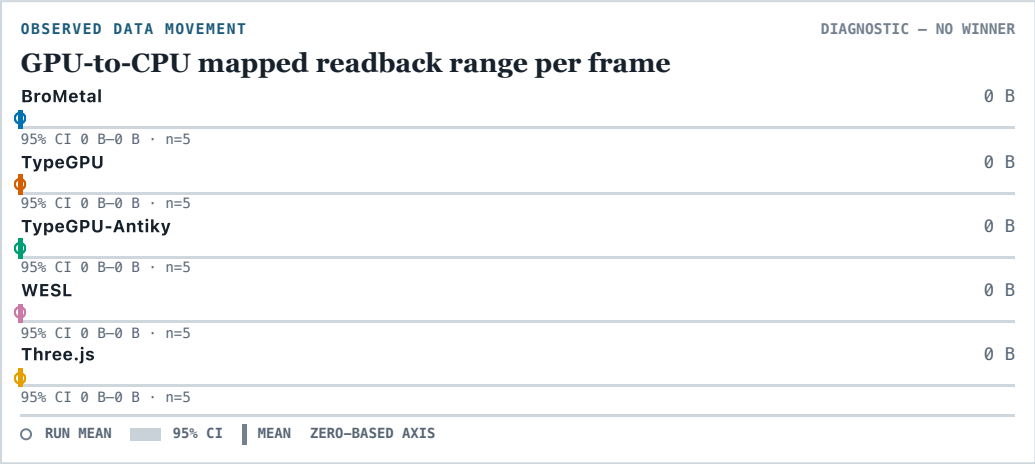
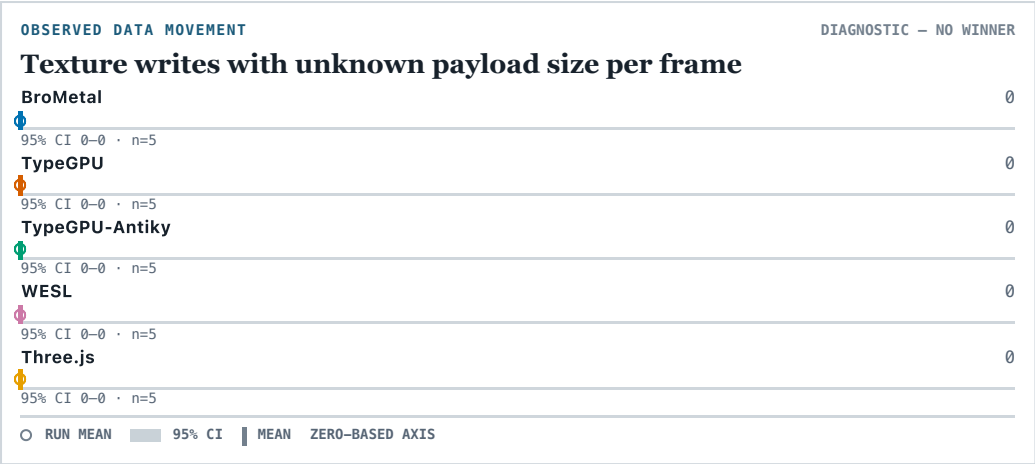
Mapped writes report CPU-writable capacity, not confirmed modified bytes. Unknown texture formats are counted separately. These are browser-visible API measurements—not physical bus transactions, cache-line traffic, driver staging, or proof of hardware occupancy. On unified-memory systems, these counters do not map directly to a physical CPU/GPU interconnect.

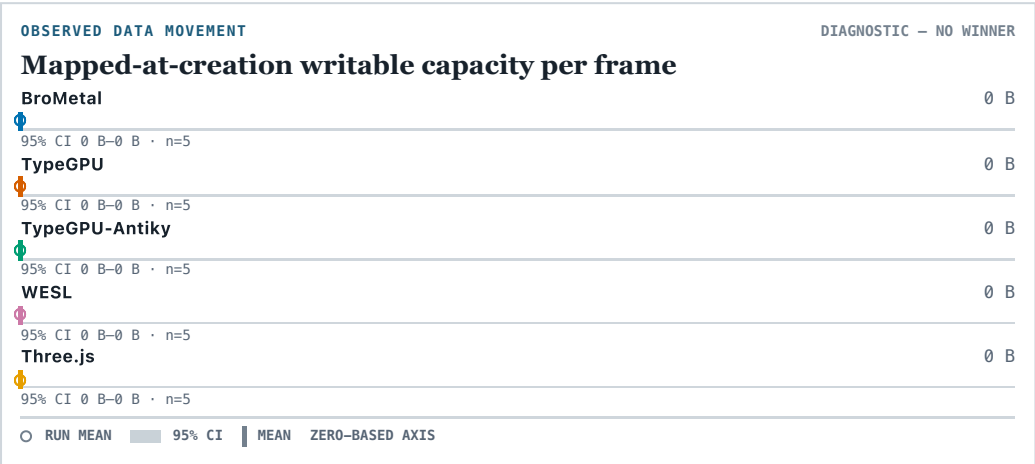
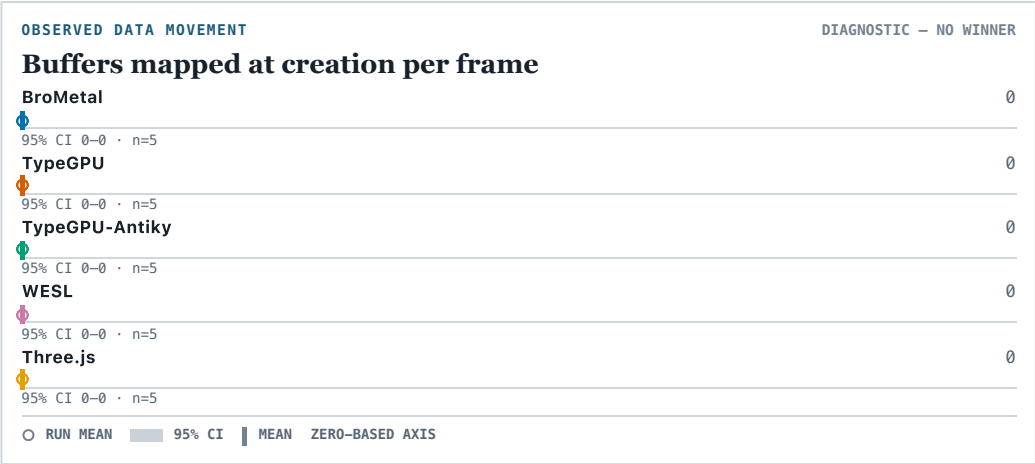
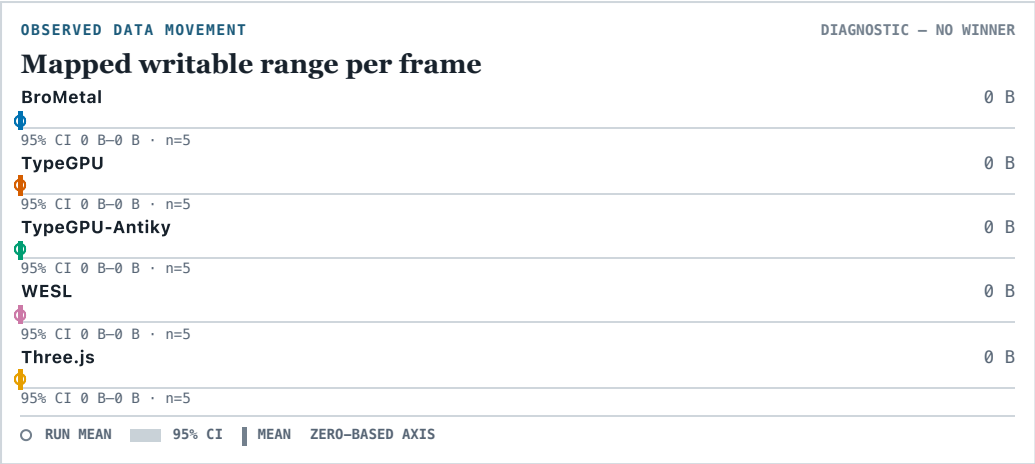
MEASURED DISTRIBUTIONS

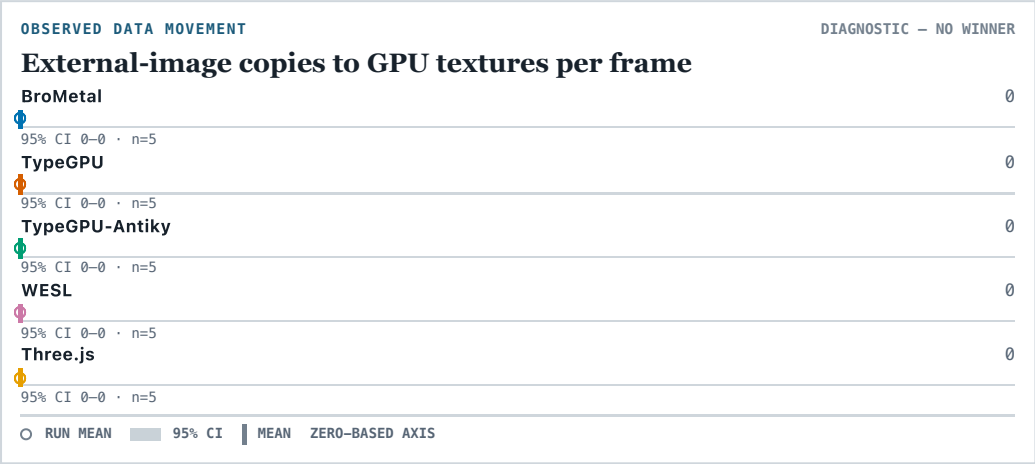
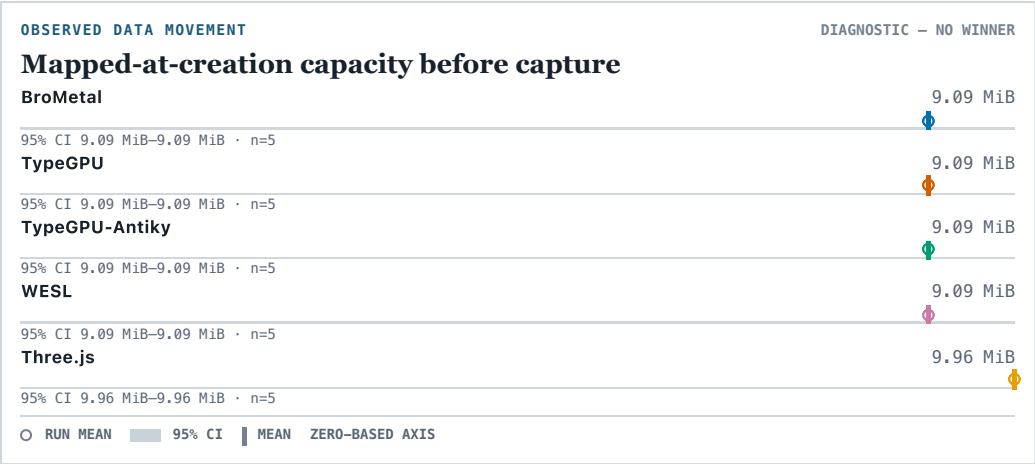
Observed data movement

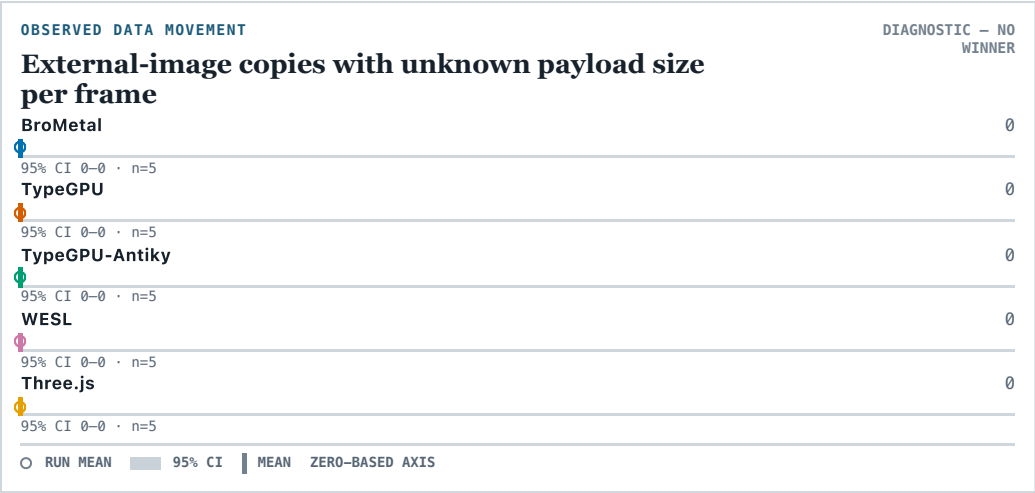
Queue writes are confirmed CPU-source API payloads; map reads are GPU-to-CPU readback requests. External-image paths, mapped-write capacity, and GPU-internal copies stay separate so the columns cannot be summed into a fabricated round-trip total.











ADDITIVE COMPOSITION

Estimated live WebGPU resources

DESCRIPTOR BYTES, NOT PHYSICAL RESIDENCY



This is the one stacked view: buffer and texture bytes are additive. Frame times and utilization are overlaid or shown independently because stacking them would imply a false total.

MEASURED DISTRIBUTIONS

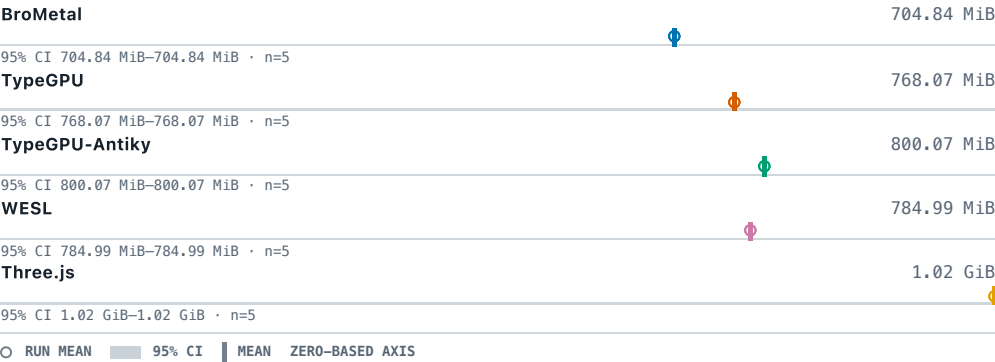
Resource and system diagnostics

WebGPU allocation is instrumented per page. Chrome process GPU time is driver-accounted; macOS device and memory counters are system-wide diagnostics, not portable hardware occupancy.

GPU RESOURCE ALLOCATION

LOWER IS BETTER

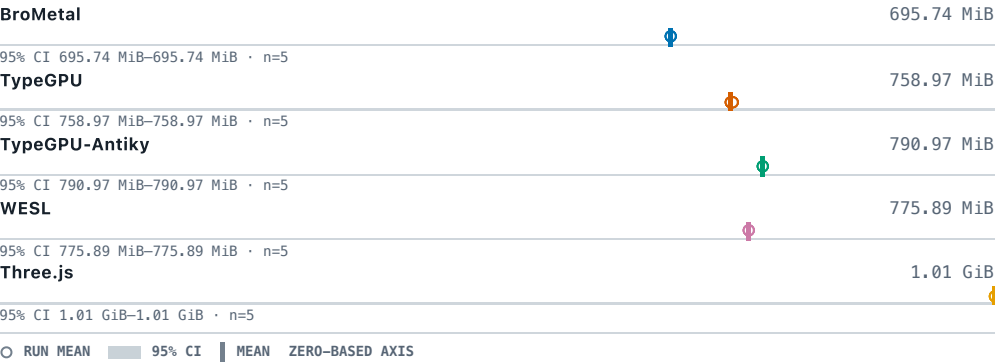
Estimated live WebGPU allocation

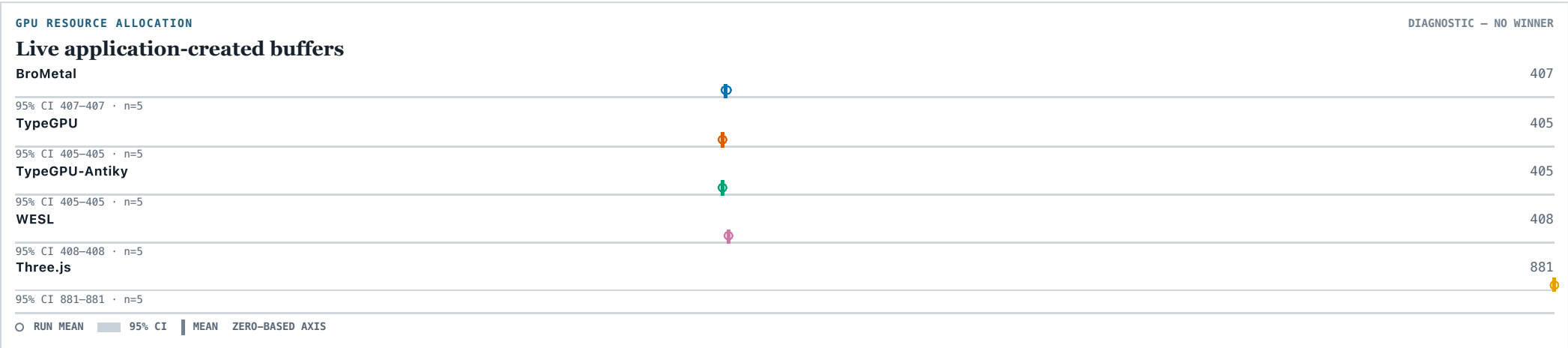


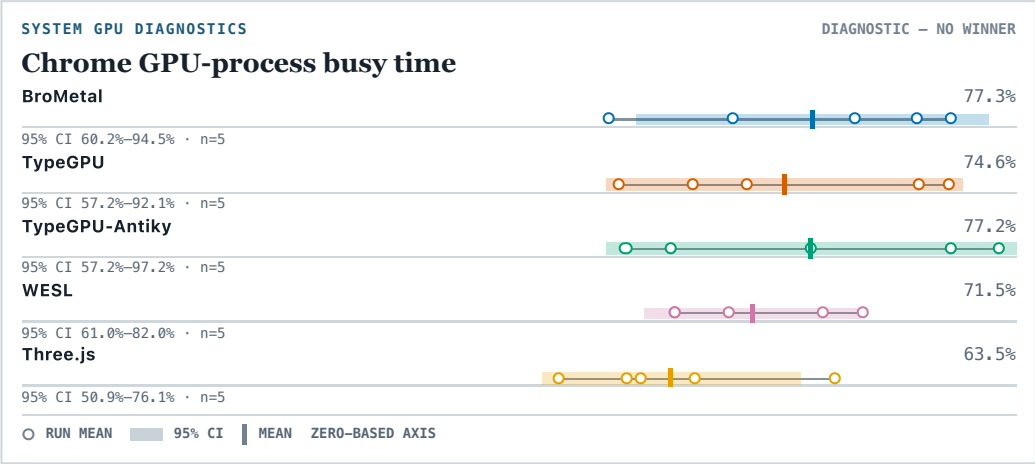
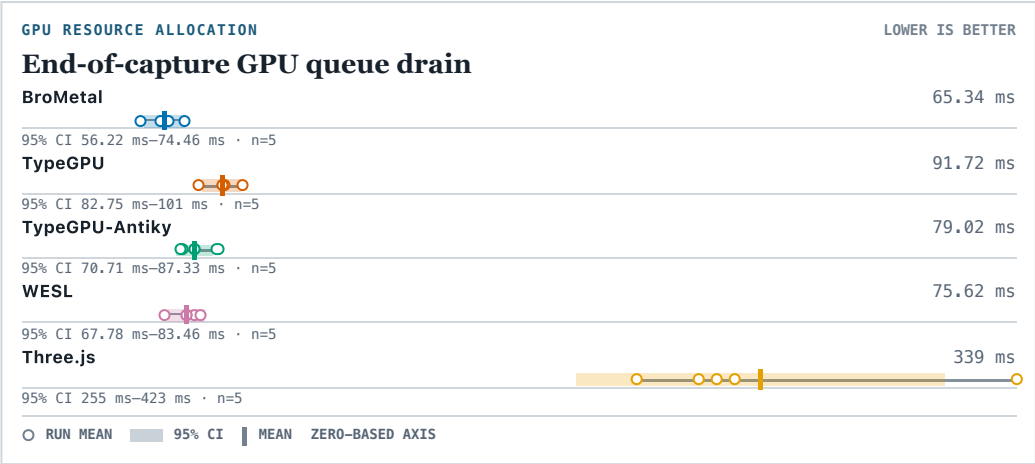
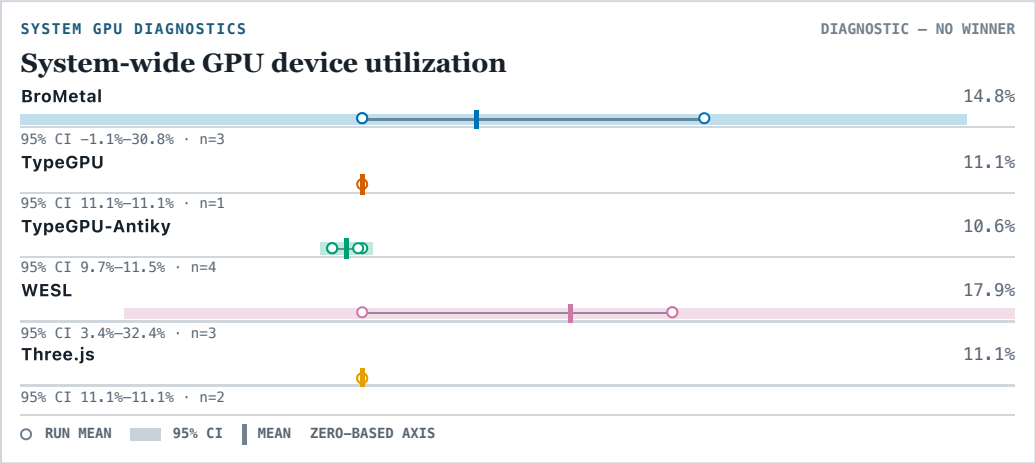
GPU RESOURCE ALLOCATION

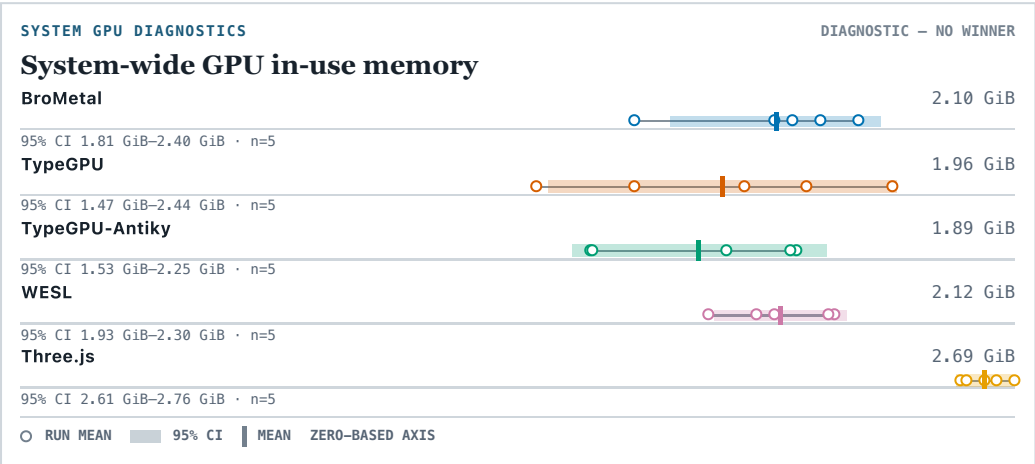
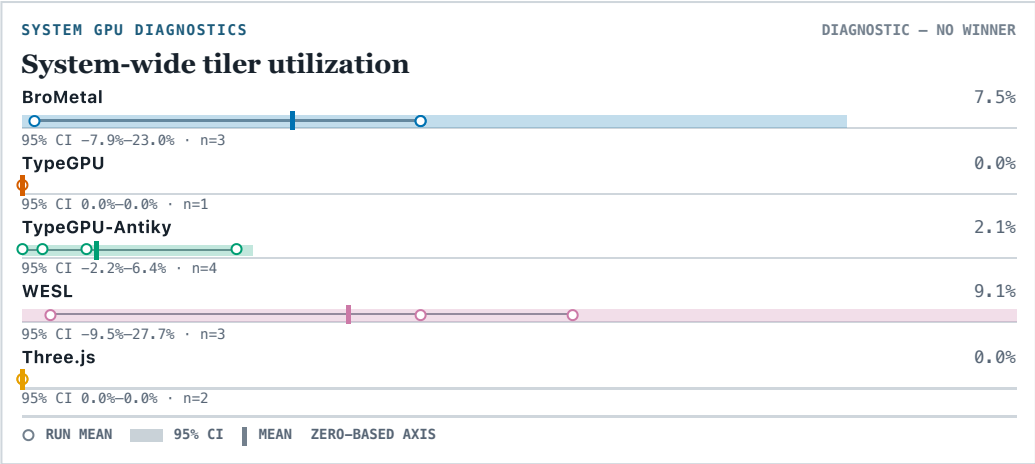
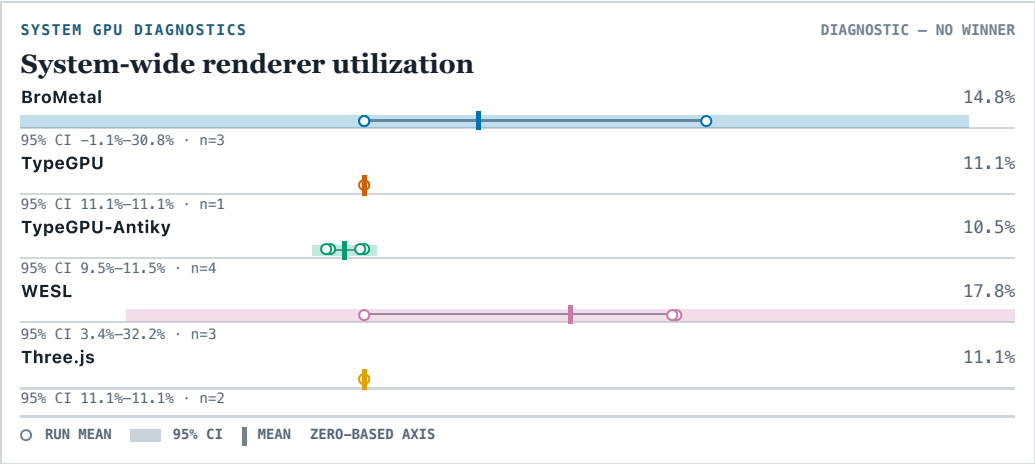
LOWER IS BETTER

Estimated live texture allocation









WORKLOAD ACCOUNTING

Scene and WebGPU operations

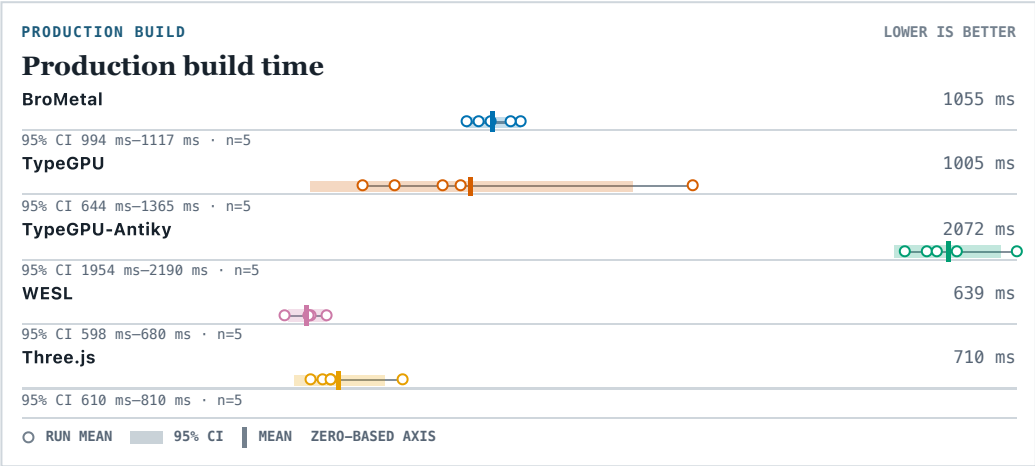
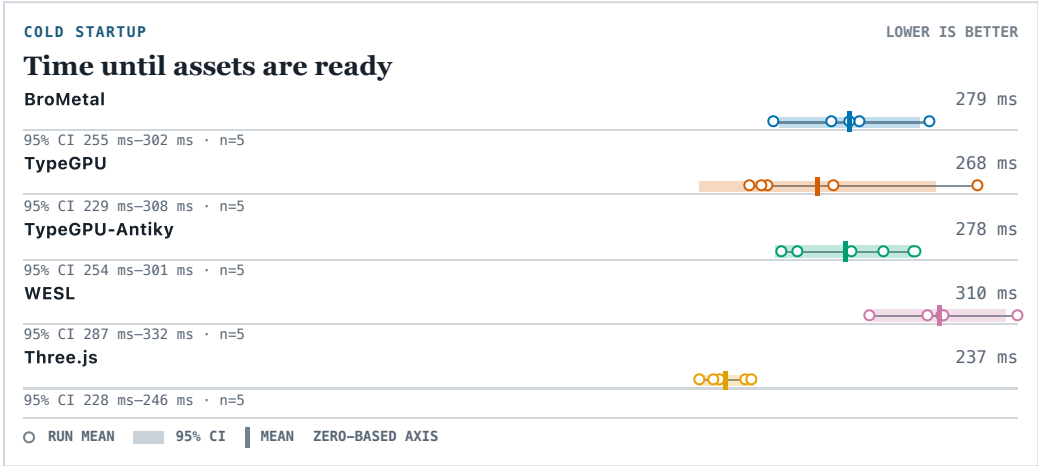
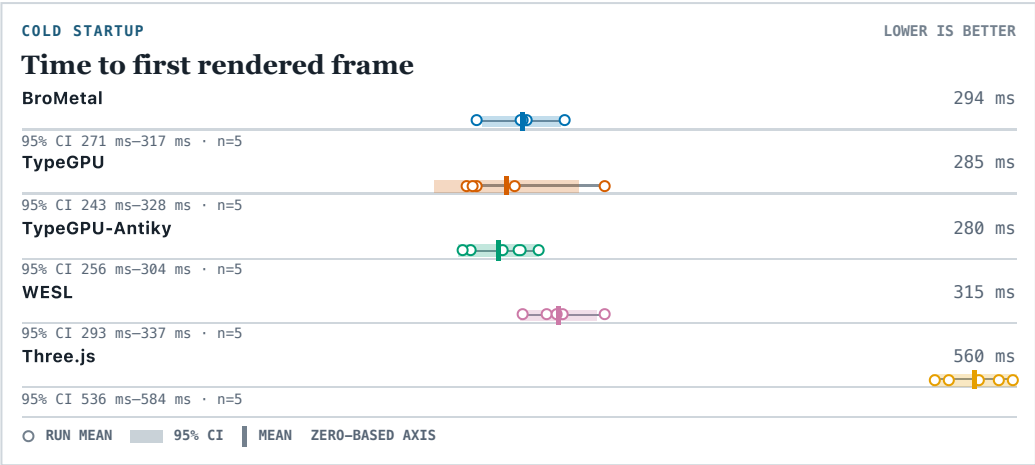
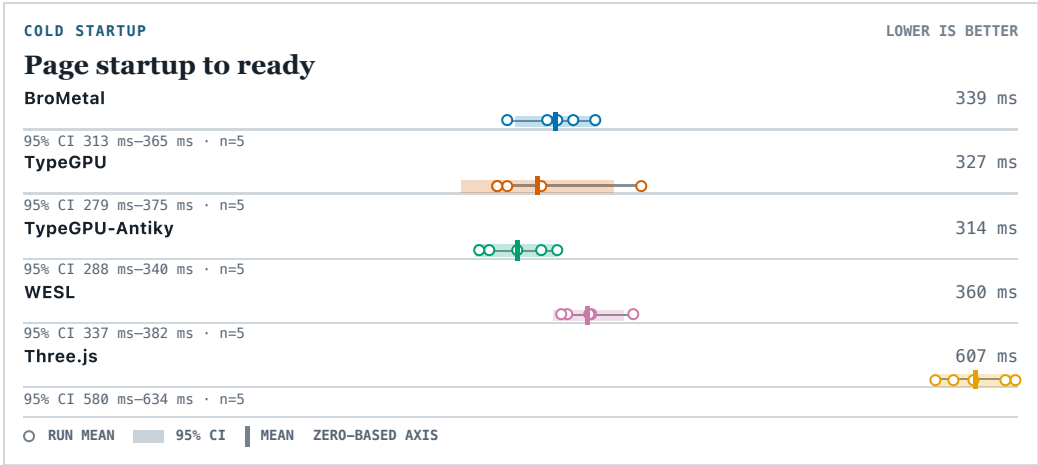
Operation counts provide context for where each renderer spends work and help explain the measured frame-rate spread.

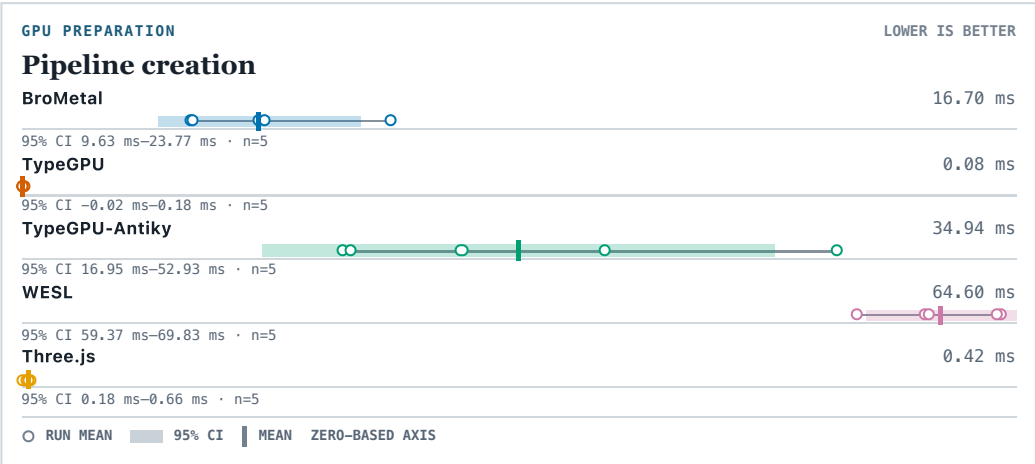
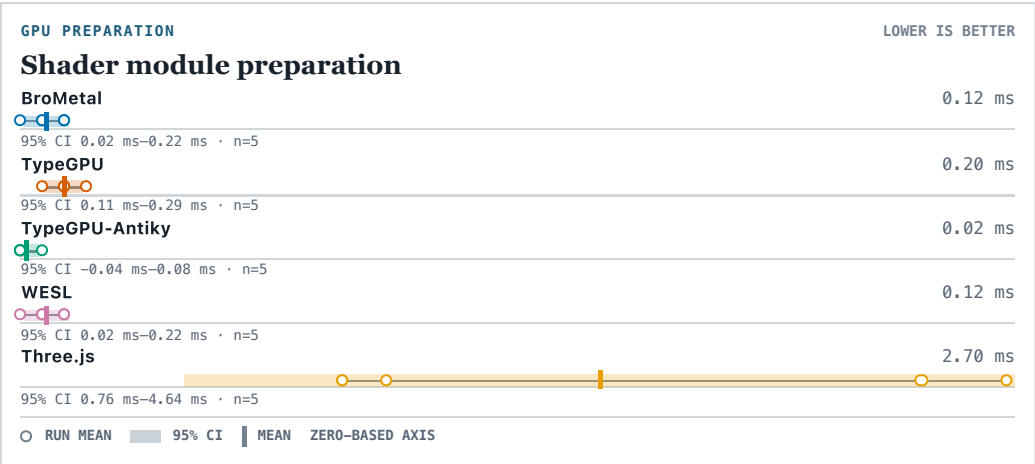
MEASUREMENT	BROMETAL	TYPEGPU	TYPEGPU-ANTIKY	WESL	THREE.JS
Render passes per frame	25	25	25	25	25
Compute passes per frame	0	0	0	0	0
Indexed draws per frame	309	309	309	309	384
Non-indexed draws per frame	22	23	22	23	21
Submitted indices per frame	2,360,403	2,360,403	2,360,403	2,360,403	3,150,924
Submitted non-indexed vertices per frame	2,499	2,502	2,499	2,502	63
Submitted draw instances per frame	736	737	736	737	1,215
Compute workgroups per frame	0	0	0	0	0
Command encoders per frame	1	1	1	1	27
Queue submits per frame	1	1	1	1	27
Submitted command buffers per frame	1	1	1	1	27
Maximum render sample count	1	1	1	1	1
Render width	2,560	2,560	2,560	2,560	2,560
Render height	1,440	1,440	1,440	1,440	1,440
Scene meshes	103	103	103	103	103
Visible scene meshes	103	103	103	103	103
Simulated scene lights	474	474	474	474	474
Analytic lights evaluated	32	32	32	32	32
Rendered particles	406	406	406	406	406

MEASURED DISTRIBUTIONS

Cold start and shipped artifacts

Fresh browser processes preserve cold-start evidence; production build and GPU preparation metrics remain separate from warmed rendering.





01 Cold start is separate Startup and pipeline preparation retain fresh-process, no-cache behavior. They are not mixed into warmed frame-rate samples.	02 Fixed observation window 5 s warmup, then exactly 10 s measured.	03 Order effects Implementation order rotates across repetitions to distribute heat and temporal drift.	04 Uncertainty is visible Dots are individual run means. Intervals use a two-sided 95% Student t interval; five runs still provide limited power.
---	--	--	--